

AI Systems Performance Engineering

Chapter 12 — Dynamic Scheduling, CUDA Graphs, and Device-Initiated Kernel Orchestration

Your Name
Applied Scientist
AWS

Internal Training for Applied Scientists on AI Infrastructure

August 15, 2026

What This Talk Is About — and Why

In plain English

Chapters 6–11 made *individual kernels* fast. But between kernels, the GPU often sits idle, **waiting for the CPU** to decide what to run next. This chapter moves the **scheduling itself onto the GPU**: work queues built from atomics, whole pipelines replayed as **CUDA Graphs**, kernels that launch other kernels — and, across many GPUs, **NVSHMEM** so devices talk without the CPU at all.

- **The problem:** launch overhead and CPU round trips leave idle gaps between kernels — microseconds that add up to real money at LLM scale.
- **The goal:** keep every SM fed, end to end, with the host **off the critical path**.
- **The tools:** atomic work queues, CUDA Graphs (host- and device-launched), conditional nodes, dynamic parallelism, NCCL + NVSHMEM.
- **The compass:** roofline analysis tells us which tool moves the needle.

This is what vLLM, TensorRT-LLM, and PyTorch do under the hood — worth understanding first-hand.

- ① **Dynamic scheduling** — atomic counters and work queues that balance uneven work
- ② **CUDA Graphs** — capture once, replay forever; dynamic updates
- ③ **Device-initiated graphs** — fire-and-forget / tail / sibling launches; conditional nodes
- ④ **Dynamic parallelism** — kernels spawning kernels for data-dependent work
- ⑤ **Multi-GPU orchestration** — peer copies, NCCL in graphs, NVSHMEM one-sided ops
- ⑥ **Roofline-guided decisions** — which technique for which bottleneck

Plain English: one theme, six tools: get the CPU out of the loop so the GPU never waits.

The Enemy: Uneven Work $\Rightarrow$ Idle SMs

- Input-dependent loops give threads **different amounts of work**: fast blocks finish, their SMs **idle** while stragglers grind on.
- By the time the longest block finishes, much of the GPU has done nothing for a while — **low achieved occupancy**, wasted cycles.
- Diagnose it: **Nsight Systems** shows the timeline gaps; **Nsight Compute**'s *SM Active %* shows cycles with zero active warps; add **NVTX** ranges to tie gaps to code.

The takeaway

Static, preassigned work only balances *uniform* workloads. For irregular work, assignment must happen **at runtime, on the device**.

Plain English: if you deal every worker a fixed stack of tasks, the lucky ones go home early while the unlucky ones work overtime — and you pay for the whole factory either way.

Atomic Counters: Fast, but Watch the Contention

- Global atomics are serviced **in the L2 cache** — fast when uncontended, but under contention threads **serialize** at one address.
- Metric:
`atomic_transactions_per_request` (Nsight Compute): ≈ 1.0 ideal; > 1.0 = stall-and-retry.
- Fix: **amortize** — one atomic per **batch**, not per item:

```
// Before: one atomic per item
int idx = atomicAdd(&queue_head, 1);
if (idx < N) process(data[idx]);

// After: one atomic per 32 items
int start = atomicAdd(&queue_head, 32);
for (int i = start; i < start + 32 && i < N; ++i)
    process(data[i]);
```

Plain English: send one runner per crew to fetch 32 tickets at once, instead of everyone queuing at the window.

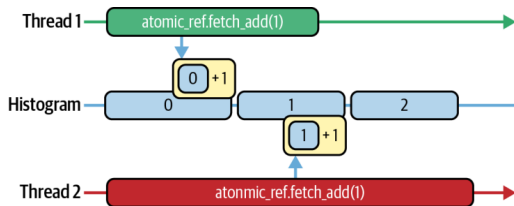


Figure 12-1. On-chip (L2) atomic adds from multiple threads

- Even batch size **8–16** kills most contention, thanks to L2 bandwidth.

Atomic Work Queues: Load Balancing on the Device

```
__device__ unsigned int globalIndex = 0;
__global__ void computeKernelDynamicBatch(...) {
    int lane = threadIdx.x & (warpSize - 1);
    while (true) {
        unsigned mask = __activemask();
        int leader = __ffs(mask) - 1;
        unsigned base = 0;
        if (lane == leader) // warp leader grabs
            base = atomicAdd(&globalIndex, warpSize);
        base = __shfl_sync(mask, base, leader); // broadcast
        unsigned idx = base + lane;
        if (idx >= N) break; // dynamic termination
        /* ... process element idx ... */
    }
}
```

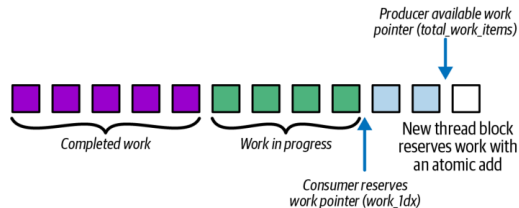


Figure 12-2. Atomic counter as a dynamic work queue across SMs

- Warps that finish early **immediately pull the next batch** — no SM idles until the queue is drained.

Atomic Queues: What You Gain, and When

	Static	Dynamic
Runtime (skewed work)	~200 ms	~ 100 ms

- **2×** in extreme imbalance; typically **10–20%** for moderate imbalance.
- For *mild* imbalance, atomic + shuffle overhead can cancel the gains — measure.

Reach for a dynamic queue when profiling shows

- warp-idle stalls / low achieved occupancy,
- timeline gaps from uneven per-task runtimes,
- `atomic_transactions_per_request` ≈ 1.0 after batching (i.e., the queue itself isn't the new bottleneck).

Plain English: no high-level PyTorch API for this — it's a custom-CUDA-kernel technique.

CUDA Graphs: Capture Once, Replay with One Call

- Launching a pipeline of kernels + copies one-by-one pays **CPU launch overhead every iteration**.
- A **CUDA Graph** records the workflow — nodes (kernels, memcpys) and dependency edges — and **replays it with one host call**.
- Dependencies known upfront $\Rightarrow$ batch scheduling, no per-step CPU handshakes, automatic overlap of independent nodes.
- Under the hood of **PyTorch**, **vLLM**, **TensorRT-LLM** (one precaptured graph per batch-size bucket).^[2,6]

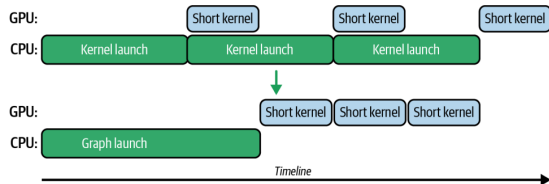


Figure 12-3. Launch timeline without (top) and with (bottom) CUDA Graphs

Plain English: record the cooking show once and press play — don't dictate the recipe every night.

Capturing a Graph: Stream Capture API

```
cudaGraph_t graph; cudaGraphExec_t instance;
cudaStreamBeginCapture(stream, cudaStreamCaptureModeGlobal);
kernelA<<<grid, block, 0, stream>>>(d_X);
kernelB<<<grid, block, 0, stream>>>(d_Y);
kernelC<<<grid, block, 0, stream>>>(d_Z);
cudaStreamEndCapture(stream, &graph);
cudaGraphInstantiate(&instance, graph, nullptr, nullptr, 0);

for (int iter = 0; iter < 100; ++iter)
    cudaGraphLaunch(instance, stream); // 1 call = A->B->C
cudaStreamSynchronize(stream);
```

- **Capture**: bracket normal stream code with `BeginCapture/EndCapture` — nothing executes during capture.
- **Instantiate** once ($\rightarrow$ `cudaGraphExec_t`), then **launch** per iteration: 1 CPU call instead of 3+.
- Partial pipelines are fine — capture just the repetitive subgraph.
- Parameters changed?
`cudaGraphExecUpdate`.

CUDA Graphs in PyTorch

```
# 1) warm-up: initialize kernels/cuBLAS lazily
_ = opC(opB(opA(X))); torch.cuda.synchronize()
static_x.copy_(X) # seed static input
# 2) capture with static (pointer-stable) buffers
g = torch.cuda.CUDAGraph()
with torch.cuda.graph(g, stream=stream):
    torch.mul(static_x, 1.1, out=static_y)
    torch.add(static_y, 2.0, out=static_z)
    torch.sqrt(static_z, out=static_w)
# 3) replay: one host call per iteration
for i in range(100):
    # static_x.copy_(new_X) # if inputs change
    g.replay()
```

- **Warm-up is mandatory:** capture won't record lazy init (cuBLAS/cuDNN); skipping it can fail the capture.
- All tensors at **fixed addresses, fixed shapes**: preallocate, write with `out=`, update inputs via `copy_` into static tensors.
- `graph_pool_handle()`: dedicated pool for pointer stability.
- No allocations, print, or RNG setup inside the capture.

Plain English: a graph is a videotape of GPU work — the actors (tensors) must stand on the same marks every take.

What Graphs Buy You — and the Fine Print

Per 100 iterations	Before	After
CPU launch calls	300	100
Host sync calls	300	0
GPU idle / iteration	~3 μ s	0
Iteration latency	1.00 ms	0.75 ms

Table 12-1 (illustrative numbers — see the book's GitHub repo^[7])

- Graphs don't make a single kernel faster — they remove **between-kernel** overhead.
- Gains grow with **many small kernels** (exactly the LLM decode profile).

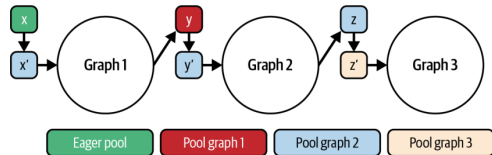


Figure 12-4. PyTorch backs graphs with static memory pools

Dynamic Graph Update: Don't Recapture, Patch

- Shapes or pointers changed? `cudaGraphExecUpdate` / `cudaGraphExecKernelNodeSetParams` patch grid/block dims, kernel params, and addresses **in the existing executable graph**.
- Takes **a few microseconds** — preserves the sub-100 μ s replay path; incompatible edits (adding/removing nodes) return an error $\Rightarrow$ recapture.

Typical inference workflow

- ➊ Capture a **template graph** at the maximum batch size (e.g., 128).
- ➋ Request arrives with batch 64 $\Rightarrow$ `cudaGraphExecUpdate` adjusts launch dims (and buffer pointers).
- ➌ Replay. Structure changed (different kernel count)? Recapture once, keep updating after.

Plain English: like editing the guest count on a standing catering order instead of renegotiating the contract every day.

Device-Initiated Graph Launch: Remove the CPU Entirely

- A running kernel can **launch a prerecorded graph from device code**:
`cudaGraphLaunch(graphExec, stream)`
inside a kernel.
- Setup: instantiate with `cudaGraphInstantiateFlagDeviceLaunch`, then `cudaGraphUpload` to GPU memory *before* any device launch.
- $\sim 2\times$ **lower launch latency** than host-side graph launch — and it stays **flat** as the graph grows in nodes or width.^[3]
- The GPU computes the decision *and* dispatches the work — no host round trip.

Plain English: the factory floor schedules its own jobs — no phoning HQ between tasks.

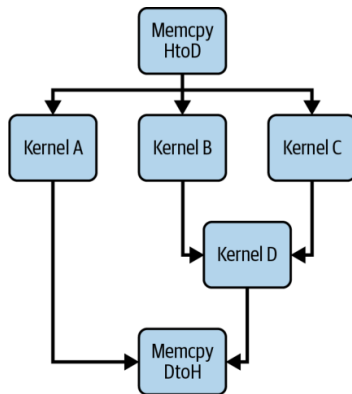


Figure 12-5. A graph of kernels and copies with dependency edges

Three Device-Launch Modes

Fire-and-forget

Child graph starts **immediately**, concurrent with the parent; parent doesn't wait. For independent side tasks. Limit: **120** per graph execution.

Tail launch

Runs **after the parent completes** — a continuation. Enables GPU-resident schedulers: a kernel can **re-enqueue itself**. Limits: **255** pending; one pending **self**-tail-launch.

Sibling

Fire-and-forget as a **peer** of the parent graph, in the parent's stream environment — runs now, without delaying the parent's tail launches.

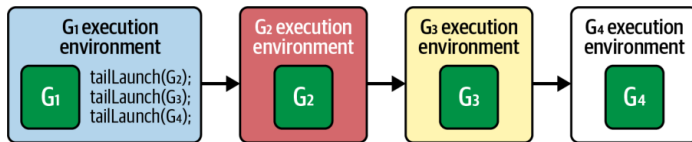


Figure 12-6. Tail launches execute one at a time, in enqueue order

- Need the child's results? **Tail**. Independent side job? **Fire-and-forget** / **sibling**.

Pattern: A GPU-Resident LLM Decode Scheduler

Atomic queue + tail launch = persistent in-kernel scheduler

- ❶ Capture the transformer block's forward pass (attention + FFN) as a **decode graph**.
 - ❷ A lightweight **persistent scheduler kernel** claims work: `atomicAdd(&queueHead, 1)`.
 - ❸ It **tail-launches** the precaptured decode graph for that item, then loops for the next.
- Every token is processed start-to-finish **on the device** — the CPU never re-enters the critical path.
 - Adapts on the fly: switch between precaptured graphs, or `cudaGraphExecUpdate` parameters for new sequence lengths / batch sizes.
 - This is the architectural idea behind modern **GPU-resident inference engines**.

Plain English: the GPU becomes its own dispatcher: take a ticket, run the model, take the next ticket — the CPU just watches the queue fill.

Conditional Graph Nodes: Control Flow Inside the Graph

- Classic graphs are **fixed at capture time** — decisions bounce back to the host.
- **Conditional nodes** embed **IF / IF-ELSE / WHILE / SWITCH** in the graph; the GPU evaluates a **condition handle** and picks the body subgraph itself.
- A device kernel sets the flag: `cudaGraphSetConditional(handle, flag)` — from a **single thread**, to avoid races.
- Nodes **nest** (WHILE containing IF): multilevel logic, zero CPU hops.
- PyTorch: **no Python API yet** — custom C++ integration required.

Plain English: the recorded show gains “choose your own adventure” buttons — and the GPU presses them.

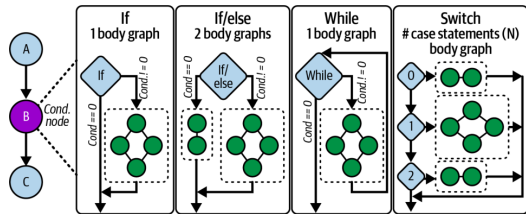
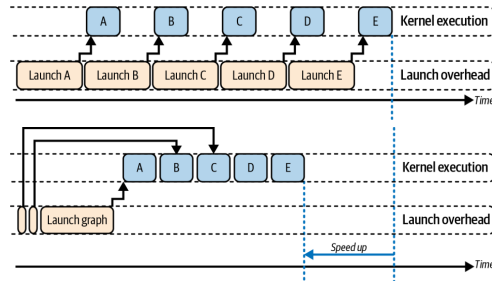


Figure 12-9. Conditional node types: IF, IF/ELSE, WHILE, SWITCH

Dynamic Parallelism: Kernels That Launch Kernels

- Graphs replay **known** pipelines. When the *shape of the work emerges from the data* — graph traversal, adaptive mesh, hierarchical reduction — you can't prerecord it.
- **DP**: a parent kernel inspects its own output and launches **child kernels** on the device (compile with `-rdc=true`).
- LLM example: most tokens take the standard path; a DP kernel tail-launches an auxiliary attention block **only for the tokens that need it**.
- Telltale profile: Kernel A → GPU idle gap → Kernel B, where the gap is the CPU deciding.



Figures 12-12 / 12-13. Host-driven gaps (top) vs. device-side launches (bottom)

Dynamic Parallelism: Numbers and Caveats

Metric (2 children)	Host	Device
Host launches	3	1
Overhead per launch	~20 μ s	~25 μ s
GPU idle during sequence	~40%	~ 5%
Total runtime	1.00 ms	0.75 ms

Table 12-2 (illustrative^[7]). Bonus: intermediates **stay in GPU memory** — better locality.

- Parent implicitly **waits for all children** — no device-side `cudaDeviceSynchronize()` needed (host syncs the stream once).
- Child launches consume **stack space** and pending-launch slots; default cap **2,048 pending** children — both raised via `cudaDeviceSetLimit`.
- Device launch $\approx$ host launch in overhead: **tiny children lose**. Fixed, repeated flow? Prefer **device-launched graphs** — they amortize.

Plain English: DP is for work you can't predict; graphs are for work you can.

Scaling Out: Overlap Communication with Compute

- Multi-GPU goal is unchanged: **hide data movement behind useful work**. NVLink is fast; it is still not HBM — overlap or plateau.
- **Peer-to-peer**: `cudaMemcpyPeerAsync(dst, dstGpu, src, srcGpu, size, commStream)` on a second stream while compute streams keep running.
- **NCCL**: nonblocking collectives (all-reduce rings/trees) saturate NVLink/NVSwitch alongside compute streams — the engine of PyTorch DDP.
- **CUDA-aware MPI**: pass device pointers to `MPI_Send`; **GPUDirect RDMA** moves data NIC→HBM over InfiniBand, no host staging.
- Context: a GB200 **NVL72** rack = 72 GPUs in one NVLink domain, up to **30 TB** unified memory — one giant coprocessor if software keeps up.^[1]

Plain English: without overlap, scaling stops the moment communication time equals compute time; with overlap, 4 GPUs can look like one 4×-faster GPU.

NVSHMEM: GPUs Read and Write Each Other's Memory

- **PGAS** model: each GPU is a **PE** (processing element) in a partitioned global address space; device code does **one-sided put/get** into a peer's memory.
- Classic **send-and-signal**:

```
// GPU 0 kernel:  
nvshmem_float_p(remote_data + 1, value, dest_pe); // put  
nvshmem_quiet(); // ensure completion  
nvshmem_int_p(remote_flag, 1, dest_pe); // raise flag  
// GPU 1 kernel:  
nvshmem_int_wait_until(remote_flag, NVSHMEM_CMP_EQ, 1);  
float val = remote_data[1]; // payload is valid
```

Plain English: instead of mailing packages through the post office (CPU), GPUs walk over and put things directly on each other's desks.

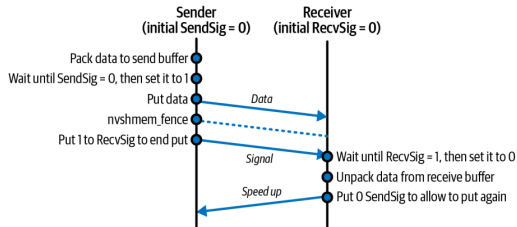


Figure 12-14. NVSHMEM one-sided communication

- No CPU, no kernel-launch overhead: a multistep exchange becomes **one hardware transaction** at near wire speed.

NVSHMEM Patterns: Pipelines, Work Stealing, Lockstep

- **Two-stage pipeline:** GPU 0 computes attention, **puts** activations to GPU 1 + signals; GPU 1's persistent kernel starts the MLP while GPU 0 moves to the next batch — handoffs **hidden behind compute**.
- **Device-side work stealing:**

```
while (true) {  
    int idx = nvshmem_int_atomic_inc(queue_head);  
    if (idx >= N_tasks) break;  
    // process tasks[idx] ...  
}
```

- **Lockstep steps:** launch one cooperative kernel on all PEs with `nvshmemx_collective_launch()` (required for device-side barriers/collectives), then `nvshmem_barrier_all()` inside.
- **Don't over-synchronize:** global barriers stall everyone on the slowest peer — prefer point-to-point signals (`nvshmem_signal_*`).
- Shines for **irregular** work: graph algorithms, dynamic load balancing, event-driven simulation.
- NVSHMEM kernels **can be captured** inside CUDA Graphs.

NCCL Inside CUDA Graphs: Deterministic Training Steps

```
cudaStreamBeginCapture(captureStream,  
    cudaStreamCaptureModeGlobal);  
forwardKernel<<<..., captureStream>>>(...);  
ncclAllReduce(sendBuf, recvBuf, count, ncclFloat,  
    ncclSum, comm, captureStream);  
backwardKernel<<<..., captureStream>>>(...);  
cudaStreamEndCapture(captureStream, &graph);  
// instantiate + upload once; then per step:  
cudaGraphLaunch(graphExec, captureStream);
```

- **Bucketed all-reduce**: split gradients across streams inside the capture — comm on stream B overlaps compute on stream A (Figure 12-15).

Plain English: everyone in the orchestra plays from the same recorded score — if one violinist improvises, the concert deadlocks.

- NCCL calls become **graph nodes**; per-step host work drops to **one cudaGraphLaunch** — less CPU load, less **launch jitter**.
- Rules: **all ranks** capture and replay the **same sequence, same communicator** — mismatch risks **deadlock** or silent corruption.
- Run **warm-up collectives** before capture to initialize communicators.
- At cluster scale these savings **compound**: tighter sync, higher utilization.

Roofline-Guided Orchestration: Which Tool, When

- **Memory-bound** (left of ridge): overlap is king — async copies, streams, concurrent kernels; **reduce bytes** with FP16/FP8/FP4. (Fusion helps only modestly.)
- **Compute-bound but under the roof**: kill launch overhead — **fusion, persistent kernels, CUDA Graphs, device-side launches** push you toward the flat ceiling.
- **Neither** (mid-chart): **increase concurrency** — multiple kernels/streams/graphs in parallel raise the aggregate point on both axes.

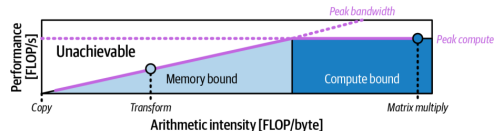


Figure 12-17. Two ceilings: memory roof (sloped), compute roof (flat)

The takeaway

Count FLOPs and bytes with Nsight Compute, plot the point, **then** pick the technique — and re-measure after every change. Roofline guides; **profiling verifies**.

Key Takeaways

- **Atomic work queues:** L2 atomics + batched increments balance irregular work on-device — up to $2\times$ in extreme imbalance, 10–30% typical.
- **CUDA Graphs:** capture once, replay with one call — typically **20–30% latency reduction** for launch-bound pipelines; static addresses required.
- **Device-side orchestration:** device-launched graphs (tail / fire-and-forget / sibling), conditional nodes, and DP keep decision + dispatch on the GPU.
- **DP vs. graphs:** DP for work that *emerges from data*; graphs for *repeated, known* flow.
- **Multi-GPU:** always overlap — peer copies, NCCL in graphs, CUDA-aware MPI, NVSHMEM one-sided ops; near-linear scaling needs the CPU off the critical path.
- **Roofline first:** memory-bound $\rightarrow$ overlap + fewer bytes; compute-bound $\rightarrow$ fewer launches; in-between $\rightarrow$ more concurrency.

Conclusion and What's Next

This chapter in one sentence

Fast kernels aren't enough — **orchestration** (queues, graphs, device-initiated launches, GPU-to-GPU memory sharing) is what keeps every SM fed and the CPU out of the loop.

- Hardware is dissolving the CPU–GPU boundary (Grace Blackwell, Vera Rubin, coherent NVLink) — but **software** still has to exploit it; these techniques are how.
- Next chapter (13): how **PyTorch** packages these ideas — streams, graphs, async ops, optimized kernels — into a few lines of Python.
- Pairs with my Chapter 6 talk: first **fill** the GPU (occupancy), then **never let it drain** (orchestration).

Questions?

References & Further Reading

- 1 C. Fregly. *AI Systems Performance Engineering* (O'Reilly, 2026), Chapter 12; NVIDIA GB200 NVL72 product page (72-GPU NVLink domain, 30 TB unified memory).
- 2 NVIDIA. *CUDA C++ Programming Guide* — CUDA Graphs, stream capture, `cudaGraphExecUpdate`, dynamic parallelism, device runtime limits.
- 3 NVIDIA Developer Blog. *Enabling Dynamic Control Flow in CUDA Graphs with Device Graph Launch* — device-initiated launches, fire-and-forget / tail / sibling modes.
- 4 NVIDIA Developer Blog. *Dynamic Control Flow in CUDA Graphs with Conditional Nodes* (CUDA 12.4+) — IF / IF-ELSE / WHILE / SWITCH nodes.
- 5 NVIDIA. *NVSHMEM documentation* — PGAS model, one-sided put/get, signals, `nvshmemx_collective_launch`; *NCCL documentation* — graph-captured collectives.
- 6 PyTorch Blog. *Accelerating PyTorch with CUDA Graphs* — `torch.cuda.CUDAGraph`, static memory pools, reduce-overhead compile mode.
- 7 Book code and benchmarks: the book's GitHub repository (per-architecture results for the illustrative tables in this deck).